If already registered, click to check your payment status

Course outline

About NPTEL

How does an NPTEL online course work?

Week 1

Week 2

Week 3

week 4

Week 5

Week 6

Week 7

Week 8

Week 9

Week 10

Week 11

Week 12

Page Rank - How does Google Work ? - Part 01

Page Rank - How does Google Work ? - Part 02

Page Rank - How does Google Work ? - Part 03

Page Rank - How does Google Work ? - Part 04

Page Rank - How does Google Work ? - Part 05

Page Rank - How does Google Work ? - Part 06

Page Rank - How does Google Work ? - Part 07

Page Rank - How does Google Work ? - Part 08

Page Rank - How does Google Work ? - Part 09

Page Rank - How does Google Work ? - Part 10

Page Rank - How does Google Work ? - Part 11

Page Rank - How does Google Work ? - Part 12

Page Rank - How does Google Work ? - Part 13

Page Rank - How does Google Work ? - Part 14

Page Rank - How does Google Work ? - Part 15

Page Rank - How does Google Work ? - Part 16

Collatz Conjecture - Part 01

Collatz Conjecture - Part 02

JOC Conclusion

Week 12 Feedback Form: The Joy of Computing using Python

Week 12 : Programming Assignment 1

Week 12 : Programming Assignment 2

Week 12 : Programming Assignment 3

Quiz: Week 12 : Assignment 12

Week 12 Solution

Text Transcripts

Download Videos

Problem Solving Session - July 2025

Week 12 : Assignment 12

The due date for submitting this assignment has passed.

Due on 2025-10-15, 23:59 IST.

As per our records you have not submitted this assignment.

In the lectures, we discussed a Python program that simulates a simplified version of the PageRank algorithm using **point distribution**. The program creates a directed graph, assigns points to nodes, redistributes those points according to outgoing edges, and compares the results with NetworkX's built-in PageRank function.

1. Graph Creation
A directed graph (DiGraph) with 10 nodes (numbered 0 to 9) is created using NetworkX.

2. Random Edge Generation
The function responsible for adding edges loops over all pairs of distinct nodes (s, t) and adds a directed edge from s to t with a **50% probability**. Self-loops are avoided by checking that $s \neq t$.

3. Initial Point Assignment
Every node starts with **100 points**. These points are stored in a list where the index corresponds to the node number.

4. Point Distribution Logic
During redistribution:
- If a node has **no outgoing edges,** it keeps all its points.
- If a node has outgoing edges, its points are **split equally** among them.

5. Iterative Redistribution
The program repeatedly redistributes points over multiple rounds. After each round, the new distribution is printed. The user can stop the process by typing #.

6. Graph Visualization
The graph structure, including nodes and edges, is displayed using matplotlib.

7. Ranking Nodes by Points
At the end, a ranking function stores final points in a dictionary (node: points) and sorts them in ascending order to find which nodes have the lowest and highest scores.

8. Comparison with PageRank
The program also calls NetworkX's built-in pagerank() function to compute the official PageRank values for the same graph and prints them for comparison with the manual simulation.

This simulation helps visualize how **importance flows through a network** and how PageRank emerges as a stable distribution of scores when point redistribution is repeated many times.

1) What type of graph is created in this program? *1 point*

- Weighted graph
- Bipartite graph
- Directed graph
- Undirected graph

No, the answer is incorrect.
Score: 0
Accepted Answers:
Directed graph

2) How many nodes are initially added to the graph? *1 point*

- 9
- Random number
- 10
- 5

No, the answer is incorrect.
Score: 0
Accepted Answers:
10

3) Which library is used to handle the graph? *1 point*

- numpy
- matplotlib
- networkx
- pandas

No, the answer is incorrect.
Score: 0
Accepted Answers:
networkx

4) In add_edges(), what is the probability that an edge will be added between two distinct nodes? *1 point*

- 75%
- 0%
- 25%
- 50%

No, the answer is incorrect.
Score: 0
Accepted Answers:
50%

5) Which condition in add_edges() ensures no self-loop is added? *1 point*

- $s \neq t$
- random.random()
- $s == t$
- $s > t$

No, the answer is incorrect.
Score: 0
Accepted Answers:
$s \neq t$

6) Which function assigns the starting points to nodes? *1 point*

- pagerank()
- rank_by_points()
- add_edges()
- assign_points()

No, the answer is incorrect.
Score: 0
Accepted Answers:
assign_points()

7) How many points does each node start with? *1 point*

- 50
- 10
- 500
- 100

No, the answer is incorrect.
Score: 0
Accepted Answers:
100

8) If a node has no outgoing edges during distribution, what happens to its points? *1 point*

- They remain with the same node
- They are doubled
- They are removed
- They are given to random nodes

No, the answer is incorrect.
Score: 0
Accepted Answers:
They remain with the same node

9) How are points distributed if a node has multiple outgoing edges? *1 point*

- Given entirely to the first edge
- Divided equally among outgoing edges
- Distributed randomly
- Not distributed

No, the answer is incorrect.
Score: 0
Accepted Answers:
Divided equally among outgoing edges

10) Which function handles the repeated redistribution of points? *1 point*

- rank_by_points()
- keep_distributing()
- pagerank()
- add_edges()

No, the answer is incorrect.
Score: 0
Accepted Answers:
keep_distributing()

11) How does the user stop the redistribution process? *1 point*

- Type exit
- Close the program
- Press #
- Press Enter

No, the answer is incorrect.
Score: 0
Accepted Answers:
Press #

12) What does nx.draw(G, with_labels=True) do? *1 point*

- Assigns weights to edges
- Runs PageRank
- Adds nodes to the graph
- Displays the graph visually

No, the answer is incorrect.
Score: 0
Accepted Answers:
Displays the graph visually

13) What data structure does assign_points() return? *1 point*

- Dictionary
- List
- Tuple
- Set

No, the answer is incorrect.
Score: 0
Accepted Answers:
List

14) Which function ranks nodes by their final point values? *1 point*

- rank_by_points()
- keep_distributing()
- pagerank()
- assign_points()

No, the answer is incorrect.
Score: 0
Accepted Answers:
rank_by_points()

15) What is stored in the dictionary inside rank_by_points()? *1 point*

- {node: points}
- {points: node}
- {rank: pagerank}
- {edge: weight}

No, the answer is incorrect.
Score: 0
Accepted Answers:
{node: points}

16) Which built-in function is used to calculate actual PageRank values? *1 point*

- nx.page_rank()
- nx.rank()
- nx.pagerank()
- nx.graph_rank()

No, the answer is incorrect.
Score: 0
Accepted Answers:
nx.pagerank()

17) What is the purpose of comparing manual distribution with nx.pagerank()? *1 point*

- To visualize the graph
- To increase node count
- To verify the manual simulation
- To remove edges without points

No, the answer is incorrect.
Score: 0
Accepted Answers:
To verify the manual simulation

18) Which library is responsible for random edge creation? *1 point*

- random
- itertools
- math
- numpy

No, the answer is incorrect.
Score: 0
Accepted Answers:
random

19) If random.random() returns 0.7, will an edge be added between two distinct nodes? *1 point*

- Yes
- No
- Only if $s > t$
- Only if $s < t$

No, the answer is incorrect.
Score: 0
Accepted Answers:
No

20) What is the total number of points in the system at the start? *1 point*

- 1000
- Depends on number of edges
- 500
- 100

No, the answer is incorrect.
Score: 0
Accepted Answers:
1000